

动态调整 GC 策略

测试用例

任务三的测试用例延续了我在任务二中自己写的 LRUCacheTest 测试用例，并在其基础上做了优化。该 LRUCacheTest 测试初始堆大小为 128MB、最大堆大小为 1GB。(即 `-Xms128m -Xmx1g`) 该 LRUCacheTest 测试将 Cache 设置的很小，仅为总写入量的 10%，这使得老区中的对象很快失效，进而频繁地触发 Mixed GC，便于观察动态调整策略对 MixedGC 效果的影响。同时以 30%的概率随机选择一部分对象长期存活于老区，避免老区对象总是很容易被清理掉导致无法触发 Full GC。

不选择老师给出的 BigRamTester 测试的原因是：该测试的过程是一次性分配大量对象直至堆将近满时才停止，不涉及 LRUCache 的换入换出，故老区对象一般都是存活的，很难触发 Mixed GC，不方便观察 MixedGC 的效果。

将测试用例的最大堆大小设置成 1g 这个相对小的值的原因是：因为 Cache 设置的很小，老区的对象很快就会失效，如果堆大小设置的太大的话将很难触发 Full GC。我希望该测试能够多次触发 Full GC，这样可以将其作为最终评估调整策略性能的一项指标。

修改思路

我的做法简单来说就是通过预测分配速率和垃圾回收速率，同时结合堆的占用情况，动态调整 G1OldCSetRegionThresholdPercent 和 G1MixedGCCountTarget 这两个 flag。

首先是这两个 flag 的含义：

G1OldCSetRegionThresholdPercent, An upper bound for the number of old CSet regions expressed as a percentage of the heap size. default 10%

G1MixedGCCountTarget, "The target number of mixed GCs after a marking cycle. default 8

- G1OldCSetRegionThresholdPercent：控制 MixedGC 回收的老年代 region 占堆总大小的百分比。如果老年代 region 的比例超过这个阈值，混合 GC 将推迟或停止运行。该参数可以防止 MixedGC 阶段过度回收老年代，避免产生过多的停顿。
- G1MixedGCCountTarget：MixedGC 阶段的目标周期数，表示期望在多少次 GC 周期内完成对老年代 (Old Generation) 的回收。

清楚了两个参数的具体含义之后，我动态调整 MixedGC 策略的思路如下：

在每次 collection 阶段结束时，根据 `_collection_set`(gc 计划回收的 region 的集合)的大小和 collection 阶段的时间计算出一个垃圾回收速率 `garbage_rate`。

与 IHOP 的预测方式类似，根据近几次的 `garbage_rate` 预测出下一次 GC 的垃圾回收速率，将该速率和老区空间的分配速率 `allocation_rate` 进行比较，如果垃圾回收速率小于老区空间的分配速率，那么应该适当增大老区垃圾回收的力度。即增加 `G1OldCSetRegionThresholdPercent` 使得 GC 可以回收更多的老年代空间，同时适当降低 `G1MixedGCCountTarget`，使 MixedGC 可以尽快完成。

反之如果垃圾回收速率超过老区空间的分配速率，则可以降低 `G1OldCSetRegionThresholdPercent`，提高 `G1MixedGCCountTarget`，进而减少 MixedGC 的暂停时间。

同时还可以综合考虑堆的占用情况，如果堆已经分配的空间加上预测的堆即将分配出去的空间超过了堆总容量，说明堆即将满，此时也应该加快老区的回收，避免出现空间不足的情况。

选择 `garbage_rate` 进行预测的原因是：垃圾回收速率在参数没有变化的情况下较难突变，IHOP 的预测模型是基于标准差和置信因子的数学模型，估计值会是一个平均值。选择垃圾回收速率这种不容易突变的量用与 IHOP 相同的预测模型进行预测时会更准确一些。

具体实践：

为了实现动态调整 MixedGC 策略的功能，我主要修改了两个文件^[1]，新增了两个文件^[2]。

其中 `g1Policy.cpp` 中主要修改了以下几点：

- 修改 `record_collection_pause_end` 方法，在其中加上了一个类用来遍历 `_collection_set` 数据结构，统计 `_collection_set` 的大小。
- 新增了一个 `update_mixedgc_prediction()` 的方法，也在 `record_collection_pause_end` 方法中调用，将记录到的用于预测的信息传递给进行预测的对象。

`g1MixedGCThresholdControlh.cpp` 和 `g1MixedGCThresholdControl.hpp` 则仿照 `IHOPControl` 的写法，实现了预测功能和根据预测的结果值调整 `G1OldCSetRegionThresholdPercent` 和 `G1MixedGCCountTarget` 这两个 flag 的逻辑。

具体的改动，可以在本仓库测试用例文件夹下查看源代码。对 `g1Policy` 相关文件的改动可以结合 `patch` 文件进行查看。

测试结果

测试 1(日志不包含 cset 信息)

指标	初始	修改后
FullGC 次数	5	4
MixedGC 次数	8	6
To-space exhausted 次数	16	11
总 GC 次数	108	105
程序运行时间	1.684s	1.658s

测试 2 (日志包含 cset 信息)

指标	初始	修改后
FullGC 次数	5	4
MixedGC 次数	10	6
To-space exhausted 次数	17	11
最小 time remaining 值	94.29ms	149.69ms
总 GC 次数	114	104
程序运行时间	1.662s	1.630s

可以发现，总 GC 次数和程序运行时间基本不变，FullGC 次数、MixedGC 次数和 To-space exhausted 次数均下降，尽管 time remaining 有所减少但仍然是一个相对健康的值。

结果分析

由于 LRUCacheTest 测试程序一直在不停地分配对象，所以和实际的生产场景不同，该测试程序的分配速率一定是超出回收速率的。从直觉上来说我们应该增大 Old CSet region threshold percent，降低 Mixed GC count target。

观察修改前后的 jtr 文件，可以发现修改后 G1GC 的 Mixed GC count target 是一直在下降的，这使得 G1GC 可以更快地完成 MixedGC，减少出现空间不足的概率。事实也是如此，可以发现修改前的 G1GC 有时在进行 MixedGC 时分成了多个周期^[3]进行，而 GC(43)、GC(44)、GC(45) 都出现了 To-space exhausted 空间不足的情况。但修改后的 G1GC 在进行 GC(42)时^[4]就没有出现这种状况。尽管 To-space exhausted 主要与 Young 区对象的新增速率有关，但仍可以认为原本的 G1GC 老区对象回收过慢增大了新生代的压力。

修改前 log^[5]:

```
[0.990s][info][gc,start    ] GC(43) Pause Young (Mixed) (G1 Preventive Collection)
[0.996s][info][gc          ] GC(43) To-space exhausted
[0.998s][info][gc,start    ] GC(44) Pause Young (Mixed) (G1 Preventive Collection)
[1.005s][info][gc          ] GC(44) To-space exhausted
[1.012s][info][gc          ] GC(45) To-space exhausted
```

修改后 log^[6]:

```
[0.934s][info][gc,start    ] GC(42) Pause Young (Mixed) (G1 Preventive Collection)
[0.951s][info][gc,start    ] GC(43) Pause Young (Concurrent Start) (G1 Evacuation Pause)
[0.959s][info][gc          ] GC(44) Concurrent Mark Cycle
```

同时，修改后的 G1GC 的 Old CSet region threshold percent 也是一直在上升的，这使得 G1GC 可以更充分地回收老年代，避免因为老年代空间不足导致触发 FullGC。事实也是如此，可以发现修改前的 G1GC 在两次 MixedGC 之间触发过一次 Full GC。而修改后的 G1GC 则没有出现过这种情况。

修改前 log^[5:1]:

```
[0.990s][info][gc,start    ] GC(43) Pause Young (Mixed) (G1 Preventive Collection)
[1.019s][info][gc,start    ] GC(50) Pause Full (G1 Compaction Pause)
[1.185s][info][gc,start    ] GC(58) Pause Young (Mixed) (G1 Preventive Collection)
```

修改后 log^[6:1]:

```
[0.934s][info][gc,start    ] GC(42) Pause Young (Mixed) (G1 Preventive Collection)
[1.053s][info][gc,start    ] GC(54) Pause Young (Mixed) (G1 Preventive Collection)
[1.062s][info][gc,start    ] GC(58) Pause Full (G1 Compaction Pause)
```

查看 cset 相关日志可以发现，动态调整的 MixedGC 选取的老区数目更多^[7]，需要的 MixedGC 的次数更少。

修改前 log^[8]:

```
[0.237s][debug][gc,ergo,cset] GC(7) Finish choosing collection set old regions. Initial: 1, opt:
[0.368s][debug][gc,ergo,cset] GC(12) Finish choosing collection set old regions. Initial: 2, opt:
[0.505s][debug][gc,ergo,cset] GC(18) Finish choosing collection set old regions. Initial: 7, opt:
[0.656s][debug][gc,ergo,cset] GC(25) Finish choosing collection set old regions. Initial: 96, opt:
[0.840s][debug][gc,ergo,cset] GC(35) Finish choosing collection set old regions. Initial: 103, opt:
[0.854s][debug][gc,ergo,cset] GC(36) Finish choosing collection set old regions. Initial: 97, opt:
[0.980s][debug][gc,ergo,cset] GC(51) Finish choosing collection set old regions. Initial: 103, opt:
[0.982s][debug][gc,ergo,cset] GC(52) Finish choosing collection set old regions. Initial: 103, opt:
[0.986s][debug][gc,ergo,cset] GC(53) Finish choosing collection set old regions. Initial: 17, opt:
[1.203s][debug][gc,ergo,cset] GC(73) Finish choosing collection set old regions. Initial: 46, opt:
```

修改后 log^[9]:

```
[0.371s][debug][gc,ergo,cset] GC(12) Finish choosing collection set old regions. Initial: 1, opt:
[0.516s][debug][gc,ergo,cset] GC(18) Finish choosing collection set old regions. Initial: 16, opt:
[0.628s][debug][gc,ergo,cset] GC(24) Finish choosing collection set old regions. Initial: 111, opt:
[0.793s][debug][gc,ergo,cset] GC(31) Finish choosing collection set old regions. Initial: 268, opt:
[0.933s][debug][gc,ergo,cset] GC(39) Finish choosing collection set old regions. Initial: 189, opt:
[1.049s][debug][gc,ergo,cset] GC(48) Finish choosing collection set old regions. Initial: 211, opt:
```

总的来说，修改后的 G1GC 能够根据预测的垃圾回收速率和堆的占用情况，动态调整 MixedGC 的策略。从测试用例的结果上来看，该动态调整策略在不怎么影响总 GC 次数的情况下，减少了 Full GC、Mixed GC 和 To-space exhausted 的次数，提高了 MixedGC 的效率，降低了出现空间不足的情况的概率。

1. g1Policy.cpp 和 g1Policy.hpp ↩
2. g1MixedGCThresholdControl.cpp 和 g1MixedGCThresholdControl.hpp ↩
3. 例如 GC(43)、GC(44)，二者都是 MixedGC ↩
4. 修改后只有 GC(42)是 MixedGC ↩
5. 任务三_初始_不包含 cset 日志_LRUCacheTest.jtr ↩ ↩
6. 任务三_修改后_不包含 cset 日志_LRUCacheTest.jtr ↩ ↩
7. 尽管修改前后 initial 的值，即要回收的老区数量一开始都为 1，但显然动态调整参数的 MixedGC 该值增长更快，可以充分利用剩余时间进行老区的回收 ↩
8. 任务三_初始_包含 cset 日志_LRUCacheTest.jtr ↩
9. 任务三_修改后_包含 cset 日志_LRUCacheTest.jtr ↩